

AI Agent Orchestration Platform — Build Plan

Context

This is the Yuno AI Engineer hiring challenge ([~/Downloads/Yuno AI Engineer Challenge.pdf](#)). We must deliver a working repo + README + recorded demo for a platform where users **create AI agents** (personality, tools, schedules, memory, limits), **wire them into collaborative workflows**, run them on a **real runtime** executing **real tools**, have them **communicate asynchronously**, and expose **at least one agent through an external messaging channel** (Telegram) for a live human conversation. Everything must run locally with a single setup command and ship with a visual web UI.

Grade weights drive priorities: **working end-to-end demo 40%** · **architecture & code quality 30%** · **UI/UX & configurability 20%** · **documentation 10%**.

Decisions made (locked):

- **Stack:** Python — FastAPI control plane + separate asyncio worker pool + LangGraph runtime + PostgreSQL + Redis; **Next.js** frontend.
- **Channel:** **Telegram** (real). **Demo path = long-polling** (`getUpdates`) — no public tunnel, the safest 40% moment. **Webhooks documented as the production path** (signature verify). Slack + WhatsApp ship as documented stubs.
- **Async inter-agent messaging: run-per-message + inbox** — `send_message_to_agent` writes a message row and **enqueues a new run** for the recipient agent (consumed from a Postgres-backed queue). *Not* "inject into a running compiled graph" (that doesn't fit LangGraph and is dropped). Static routing still uses outer-graph conditional edges.
- **Scope: Full brief, everything built** — but the harness is built **thin core first, then deepened** (see Scope & sequencing). Nothing is cut by default; the deep parts are the first cut candidates if the demo/UI slip.
- **Memory: extremis wired for real** as a selectable per-agent `ExternalMemoryStrategy` alongside Buffer / Summary / ChannelScoped — **shipped in docker-compose** and must **gracefully degrade if offline** (so "single command" + offline replay still hold).

New project location: `/Users/ashwanijha/yuno-orchestrator` (git repo on `main` ; plan + architecture in `docs/`). Absolute paths here are local-dev; a fresh checkout (e.g. Ultraplan's cloud at `/home/user/repo`) uses the repo root — treat paths as relative to repo root. This file is the execution plan; `docs/architecture.md` is the canonical reference.

Review-driven changes (incorporated)

Async messaging → run-per-message+inbox (drop in-graph injection); schema reconciled (`channel_bindings.workflow_id` , `agents.default_workflow` , `agents.harness` JSONB); workflow graph stored **only** in `workflow_versions` (+ `workflows.current_version` pointer); Telegram **long-polling** for the demo; extremis **in-compose + graceful degrade**; harness **thin-first** sequencing; **thin UI vertical slice pulled forward** (Phase 4); run queue **at-least-once**

(Postgres pending-poller / Redis Streams); `require_approval_for` / `PAUSE` marked **designed-not-built** (needs the stubbed `human` node); recorded image calls **reference** `media_assets`, not inline base64.

Two invariants enforced in code

1. **Agents are config (DB rows); the runtime is generic code.** Adding an agent = a row; adding a workflow = a `graph` JSONB; never new Python.
2. **Every side effect is a Postgres row first, then a Redis publish.** Redis is transport, Postgres is truth.

System topology

- **Next.js UI (3000)** — Agent/Workflow CRUD, React Flow builder, run timeline + live monitor (WebSocket), eval pages.
 - **FastAPI control plane (8000)** — REST, WebSocket gateway, channel webhooks, scheduler.
 - **Worker pool (asyncio)** — LangGraph outer (workflow) + inner (ReAct agent) graphs, tool runtime, all LLM calls flow through the **harness**.
 - **Redis** — pub/sub (live UI), rate limits, agent inbox notifications. **Run queue is at-least-once** (Redis Streams consumer group or a Postgres pending-row poller over `runs WHERE status IN ('pending','running')`) so a worker crash between dequeue and commit re-delivers — consistent with "Postgres is truth" and the §14 heartbeat recovery. Naive `LPOP` (at-most-once) is rejected.
 - **Postgres** — source of truth (all tables below).
 - **code-runner** — separate no-network container for sandboxed `python_exec`.
 - **extremis** — memory server, in `docker-compose`; the `ExternalMemoryStrategy` degrades gracefully (falls back to buffer/summary, logs a warning) when unreachable.
-

Subsystem A — Workflow creation (where the "platform" value lives)

Three creation paths, one artifact

All three produce the **same** `workflows` row with the same `graph` JSONB. The builder is just a UI over the same API tests and templates use.

1. **Pre-built templates** — shipped in code, loaded by `scripts/seed.py`; users clone → edit. (Rubric requires ≥2.)
2. **Visual builder** — React Flow canvas; primary creation path, most UI effort.

3. **JSON import / POST /workflows** — falls out for free if the schema is right; used by templates, tests, git-versioned workflows.

The graph schema (the contract everything depends on)

Top-level: `version, name, description, entry_node, variables, nodes, edges`.

- **variables** — typed input signature (`{type, required, default}`). Manual trigger → form generated from this; channel trigger → message becomes a default variable. No more black-box "some text" workflows.
- **node input_mapping** — JSONPath-style slice of state each node needs (`$.artifacts.x` , `$.variables.topic`). Makes agents reusable across workflows.
- **node output_key** — where the node's output lands in `artifacts` ; downstream nodes reference it explicitly. Clean state flow, no implicit message-passing.
- **node config_overrides / harness_overrides** — per-node overrides of agent defaults (e.g. `max_iterations` here vs elsewhere) without cloning agents.
- **edge condition + priority** — DSL expression; edges from the same source evaluated in priority order, first match wins; unconditional edges are priority- ∞ fallback → `END` if none match. Avoids ambiguity without forcing mutually-exclusive conditions.

Node types

`agent` (run inner graph), `condition` (pure routing, no LLM), `transform` (deterministic state map), `human` (pause for approval, resume on UI action), `parallel` (fan-out N children, join), `channel_in` (await inbound), `channel_out` (send without an agent).

- **Ship now:** `agent` , `condition` , `channel_out` .
- **Stub (schema admits them, executor returns not-implemented):** `transform` , `human` , `channel_in` .
- **Stretch (week 2 if time):** `parallel` — the differentiator most candidates won't have.

Dynamic outer-graph compilation (`runtime/outer_graph.py`)

The outer LangGraph is **generated from the workflow JSON at run start**, not handwritten per workflow. Build `StateGraph(WorkflowState)` ; add one node per graph node via a per-type executor (`AgentNodeExecutor` , `ConditionNodeExecutor` , `ChannelOutExecutor`); group edges by source — single unconditional edge → `add_edge` , otherwise `add_conditional_edges` with a priority-sorted router that evaluates DSL conditions and falls back to `END` . ~150 lines. **Compile per run, not per workflow** (compilation is single-digit ms; caching invites state-sharing bugs).

Validation (`runtime/validation.py` , canonical in Python; TS copy is a UX nicety — write twice, don't share)

Reject a workflow when: entry node missing/undeclared; node references nonexistent `agent_id` ; node unreachable from entry; non-terminal node with no outgoing edges; cycle with no termination condition on any edge; condition references unknown variables/artifacts;

`input_mapping` references an artifact produced later (outside cycles); two edges from one source overlap at the same priority. Runs **client-side** for live builder feedback and **server-side** on save.

Template system (`backend/app/templates/`)

A template is a dict with `template_id, name, description, required_agents[] (role_key + default_config), graph` — graph uses **role_key s, not agent UUIDs**. Cloning: create agents from `default_config` (or map role_keys to existing agents), substitute role_keys → agent_ids, create the workflow row, open the builder. **Adding a template = one file in `templates/` + one line in `templates/__init__.py`** ; documented in `docs/adding-a-template.md` with a worked example.

- **Templates to ship:** `market_intel.py` (Researcher → Analyst → Critic **feedback loop** → Briefer → Telegram) and `personal_assistant.py` (Telegram-bound Router → Specialists, ChannelScoped/extremis memory, scheduled digest).

Versioning & edit semantics

The graph lives in `workflow_versions` **only** (immutable rows); `workflows` holds metadata + a `current_version` pointer (no `workflows.graph` column — avoids the dual-write drift hazard). Editing creates a new `workflow_versions` row and bumps the pointer. In-flight runs reference `(workflow_id, version)` and continue against the **old** version; new runs use `current_version`. UI: version dropdown, old versions load read-only with "Restore as new version"; "Re-run with same inputs" uses the run's recorded version. The answer to "what if I edit a workflow while it's running?"

Async inter-agent messaging (run-per-message + inbox)

`send_message_to_agent(recipient, content)` is a tool that, in one transaction, writes a `messages` row (`recipient_agent_id` set) and **enqueues a new run** for the recipient agent (its inbox). The recipient's run is picked up by the same at-least-once queue as any other run. This satisfies "agents communicate asynchronously" cleanly: every message is a row, every handoff is a visible run on the timeline, and there is **no attempt to push a message into an already-compiled, in-flight LangGraph** (which LangGraph doesn't support). Static, author-time routing still uses outer-graph conditional edges (the common case); this tool is the dynamic-dispatch escape hatch with a per-workflow recipient allowlist.

Multi-turn channel continuity

Each inbound Telegram message triggers a **new run** (the `channel_in` "await inbound mid-run" node is stubbed). Conversation continuity therefore rests entirely on `ChannelScopedMemory` (keyed by `channel_external_id`), not on in-run waiting — make this explicit in docs, since "live human conversation" can read like it needs a long-lived in-run loop. It doesn't.

Channel ↔ workflow binding (what makes the system cohere)

Inbound Telegram message → `/webhooks/{channel_id}` → `parse_webhook` → lookup `channel_binding` by `(channel_id, external_id)` :

- binding has `workflow_id` → trigger that workflow with the message as input variable;
- else agent's `default_workflow` if set;
- else a **synthetic single-node workflow** (not persisted) that just runs the agent's inner graph and replies — the "talk to an agent" experience. The platform feature is binding a channel to a **multi-agent** workflow (Personal Assistant template does this). Don't hardcode "one bot per agent."

Visual builder (React Flow) — four panels

Left **palette** (drag agents from `agents` table; node primitives; pre-wired "patterns" = graph fragments). Center **canvas** (custom renderers: agent shows name/role, condition shows expression, channel shows binding; edges label conditions). Right **inspector** (context-sensitive node/edge config; **CodeMirror DSL editor** with syntax highlight + autocomplete for `artifacts.*` / `variables.*` / `iteration_count`). Top bar: **Save** (new version), **Test Run** (variables modal → live execution highlighted on canvas), **Versions** (history + diff), **Validate** (static analysis with inline warnings).

Workflow-specific tests

`test_graph_validation` (table-driven, ~30 invalid graphs → right error code);
`test_conditional_routing` (branches exercised, assert path via `steps` rows);
`test_workflow_versioning` (run on v1 survives edit to v2; replay still uses v1).

Subsystem B — The unified harness (the most defensible layer)

Build order (thin-first): the thin core — `HarnessExecutor` + `Anthropic` + `Stub` + `Replay` providers + `CostCap` + `Trace` interceptors + `JSONSchema` validator — is built early (Phase 2) and is **never cut**; it's what makes the runtime real and the demo deterministic. The **deep parts** — `Bedrock` provider, `CompositeJudge`, `CitationValidator`, script `include: /Jinja2` composition, the full eval UI — are deferred to Phase 9 and are the **first cut candidates** if the demo (40%) or UI (20%) slip. This re-sequences the reviewer's "harness is over-scoped" concern without abandoning the locked "everything built" goal.

Thesis

Everything that happens around every LLM call is one lifecycle observed at different injection points: build request → call (retries/timeouts/provider quirks) → record → validate → replay → judge. **Production, test, eval, replay are configurations of one runtime, not separate systems.**

No `if testing:` anywhere — test mode is `provider=StubProvider`, demo mode is `provider=ReplayProvider`, eval mode adds an interceptor.

Core abstraction (`harness/call.py` , `harness/executor.py`)

`HarnessedCall` is the per-invocation transaction object: identity (call/run/step/agent ids), `request: LLMRequest`, resolution (`provider` , `cost_model` , `validators[]` , `interceptors[]`), result (`response` , `attempts[]` , `validation_results[]`), and observation hooks (`events` , `trace_context`). `HarnessExecutor.execute` runs six phases: (1) interceptor `before` (block/modify), (2) execute with retry on transient + validation failures, (3) validate, (4) success normalize, (5) interceptor `after`, (6) **transactional persist then** `events.emit`. The inner graph uses the harness from day one — no retrofit.

Providers (`harness/providers/` , the ONLY layer that knows provider shapes)

Protocol: `complete` , `stream` , `estimate_tokens` , `cost_model` . **Core (Phase 2): Anthropic + StubProvider + ReplayProvider.** OpenAI follows when multimodal lands; **Bedrock is deferred** (Phase 9, first cut candidate). Adapters: auth via env, tool-call format translation, system-prompt placement, structured-output APIs, streaming normalization, 429/ `Retry-After` . **StubProvider** — deterministic, YAML `Script` resolved by `agent_id / call_index / content_contains / messages_hash`, first-match-wins, latency + error injection. **ReplayProvider** — replays recorded real calls in sequence with original latency × `speed_factor`. **Discipline: never write** `if provider == "anthropic":` **outside** `providers/anthropic.py` — extend `LLMRequest / LLMResponse` with optional fields instead. Adding Gemini/Bedrock = implement the provider, register; nothing else changes.

`estimate_tokens` must be conservative (round up). The \$0.10 cost-cap demo trips *before* the call via `run.total_cost + estimate > cap`; an under-estimate lets the call through and the breaker fires late. Bias the estimate high so the breaker trips predictably on camera.

Scripts (`tests/scripts/*.yaml`)

First-class, checked-in, diffable, generatable from real runs (`harness script generate <run_id>`). The seam that makes the test harness usable. **Jinja2 templating + include: composition are deferred** (Phase 9, first cut) — start with plain literal scripts.

Validators (`harness/validators/`) — pass / fail / fail-with-retry

Core: `JSONSchemaValidator` (reinject error + schema, `retry ≤ 2` — recovers ~90% of malformed output), `MaxLengthValidator` (truncate, log), `ToolCallValidator` . `ContentSafetyValidator` (redact, no retry) with channel binding. `CitationValidator` **deferred** (first cut). Validator config lives in `agents.harness.validators` (the harness JSONB, not `guardrails`); adding one = a class + a config entry.

Interceptors (`harness/interceptors/`) — cross-cutting before/after

`CostCapInterceptor` (the demo circuit breaker — block when `run.total_cost + estimate > max_cost_per_run_usd` , return synthetic budget-exceeded response),
`IterationCapInterceptor` , `PIIRedactionInterceptor` (when bound to external channel),
`EvalRecorderInterceptor` (eval mode only), `RecordingInterceptor` (record mode only),
`TraceInterceptor` (OTel span per call). The seam where peekr / Langfuse plug in later — adding observability = one interceptor.

Five modes, one architecture (README table)

Mode	Provider	Interceptors	Use
live	Anthropic/OpenAI/Bedrock	Trace, CostCap, IterationCap, PIIRedact	production
record	real	live + Recording	capture demo / build fixtures
replay	Replay	Trace	deterministic demo + tests
stub	Stub	Trace	unit/integration tests
eval	live or replay	live + EvalRecorder	quality measurement

Same outer/inner graph, workflows, tools — only harness config changes. This table answers "how do you test in CI / measure quality / record the demo?" in one breath.

Recording & replay (`harness/recording/`)

`RecordingInterceptor` writes every completed call to `llm_recorded_calls` ; `ReplayProvider` reads them back in `sequence` order. `LLM_MODE=record RECORDING_NAME=demo_market_intel make run` → go through the flow → auto-saved; `LLM_MODE=replay ...` → free, deterministic, offline demos. Roundtrip test asserts replayed final state is byte-identical to the recorded run. **Image calls in recordings store a `media_assets` reference, not inlined base64** — keeps recordings small and the byte-identical assertion stable.

Eval framework, on the same primitives (`harness/eval/`)

An eval is a run with eval interceptors + a downstream judge — `execute_target` is the **same code path production uses**, so evals catch real regressions. Tables: `eval_datasets`, `eval_examples`, `eval_runs`, `eval_results` . Judge protocol returns `{scores{criterion→0..1}, rationale, passed, cost_usd}` . **Core:** `ExactMatchJudge` (free) + `LLMJudge` (itself a `HarnessedCall` — traced/cost-tracked/replayable). **CompositeJudge** **deferred** (first cut). Runner: bounded-parallel (`Semaphore(5)`), per-example execute-then-judge, emit live events. **UI is minimal** (a results `Table` + link to the actual run); the richer eval

pages/sparklines are deferred. Datasets: 10 examples per template, rubric-judged. **All of eval is Phase 9 / deferred** — it scores zero rubric points directly; build only if demo + UI are solid.

Harness CLI (`harness/cli.py` , ~300 lines over REST)

`harness record start/stop/list/show` , `harness eval run/status/diff` , `harness script generate/validate/run` , `harness providers list/test` , `harness cost summary` .

Recording/eval/script-gen are operational, not UI, workflows; pulling up a terminal in the walkthrough is more compelling than clicking.

Config hierarchy (`harness/config.py: HarnessConfigResolver`)

env defaults (`LLM_MODE` , `LLM_PROVIDER_DEFAULT` , `LLM_RECORDING_NAME` , `LLM_SCRIPT_PATH`) < per-agent `harness{max_attempts, retry_on, validators[], interceptors[]}` < per-node `harness_overrides` . Resolution in one place.

Harness tests

`test_executor` (retry on rate-limit/validation, fatal propagation, interceptor ordering),
`test_stub_provider` , `test_replay_provider` , `test_validators` (table-driven),
`test_cost_models` (Decimal not float), `test_recording` (record→replay roundtrip byte-identical), `test_eval_runner` (end-to-end against stub target).

Subsystem C — Multimodal (images real, video designed-not-built)

Text is the baseline; **images are built end-to-end**, video is parsed + documented. Unsupported media → **graceful text reply** ("I can't process that media type yet"), never a silent failure.

- **Channels (`telegram.py`).** `parse_webhook` handles `photo / document(image/*)` updates: resolve `file_id → getFile → download bytes` , emit `Attachment{type, mime, file_id, bytes|url}` on `InboundMessage` . `voice / video /other` → `Attachment{type=unsupported}` so the runtime can issue the graceful reply. (Voice transcription explicitly deferred.)
- **Harness (`call.py` , providers).** `LLMRequest.content` becomes **typed content blocks** (`TextBlock | ImageBlock{mime, data}`), not a bare string. Anthropic + OpenAI providers encode images natively (base64/image_url); Bedrock/Stub/Replay pass through. A capability flag (`provider.supports_images`) gates routing; if an agent's model can't see images, the prepare node substitutes a placeholder + sets the graceful-reply path. **Discipline holds:** image encoding lives only inside each provider.
- **Inner graph `prepare` node.** Builds multimodal content from `InboundMessage.attachments` ; drops/annotates unsupported types and flags the run so `send_to_channel` returns the graceful message.

- **Persistence.** New `media_assets` table (id, run_id, message_id, channel_id, type, mime, storage_ref, bytes_or_url, created_at); `messages` gains a nullable `attachments JSONB` referencing asset ids. Media stored on a local volume (or Postgres large object for the demo), referenced — not inlined into `messages.content`.
- **UI timeline.** Render image thumbnails inline on the message/step; click → full view. Outbound images supported via `send_to_channel`.
- **Video (designed-not-built).** `parse_webhook` already yields the attachment; document the implementation path: ffmpeg frame-sampling → image blocks, or Whisper transcription, or a `GeminiProvider` for native video. Executor returns the graceful reply for now.

Subsystem D — Frontend design system & component plan

Design direction (two skills, scoped)

Use **shadcn discipline** as the system language for the **dense functional surfaces** (builder, timeline blocks, config forms, tables) — restrained, token-based, consistent; this is 90% of the app and where the rubric's UI/UX + configurability points live. Borrow Anthropic's **frontend-design** skill as a *direction lens* for the **identity layer + hero moments** only — the dashboard landing, overall character, and especially the **run timeline** (orchestrated staggered reveal as blocks animate in). Pick **one** bold-but-legible differentiator (a distinctive display font for headings + Geist Mono for metrics, one saturated accent, a staggered load animation) to avoid the "generic AI dashboard" look. **Do not** apply asymmetry / grid-breaking / overlap to data tables or the inspector — legibility wins there.

Stack: Next.js App Router + **shadcn/ui** (`new-york` style, `--base radix`) + Tailwind v4 + React Flow + CodeMirror. `npx shadcn@latest init -d` then **fix the Geist font circular-reference** (literal `"Geist"` / `"Geist Mono"` names in `@theme inline`, font vars on `<html>`). Wrap root in `TooltipProvider` + `next-themes` (dark default).

Design tokens

- **Style/mode:** `new-york`, **dark mode default** (developer/admin console). Base palette **zinc**, one accent via `--color-primary`.
- **Type:** Geist Sans for UI; **Geist Mono for all metrics/IDs/costs/timestamps/latency/token counts** (run ids, `$0.043`, `12.4s`, tokens).
- **Radius:** default `0.625rem`. **Density:** compact on data-dense surfaces (`gap-4` / `p-4` / `text-sm`), comfortable on config pages.
- **Custom status tokens** (added alongside shadcn defaults in `@theme inline`): run/step status — `status-running` (accent), `status-completed` (green), `status-failed` /cost-breaker (`destructive`), `status-pending` (muted), `status-paused` (amber); plus per-agent-role

hues for builder nodes + timeline rows. Surfaces always from tokens (`bg-card` , `text-muted-foreground` , `border-border`), never ad-hoc hex.

App shell

Left sidebar nav (Dashboard, Agents, Workflows, Runs, Channels, Evals) + top bar (env/health badges, `Command` palette Cmd+K, theme toggle). Built from `Sheet` (mobile) + `Separator` + `Button` + `Badge` .

Surface → component mapping

1. **Schema-driven agent/workflow config forms** — RJSF widgets mapped to shadcn primitives (`Input` , `Textarea` , `Select` , `Switch` , `Slider` for temperature, `Badge` -based multiselect for tools). `Tabs` per group (Identity · Model · Tools · Memory · Guardrails · Harness) inside `Card` s; sticky save bar; `Sheet` for quick-edit from a list; `AlertDialog` for delete; version `DropdownMenu` ("Restore as new version"). Empty/loading/error via `Card` + `Skeleton` + `Alert` .
2. **React Flow workflow builder** — *Top bar*: `Button` group (Save · Validate) + `Dialog` (Test Run: variables form → live highlight) + `DropdownMenu` (Versions + diff). *Left palette*: `ScrollArea` + searchable `Command` + draggable agent/node cards (`Card` + `Badge` role color). *Canvas*: React Flow with custom node renderers styled from tokens (agent node = `Card` with name/role `Badge` ; condition node shows expression in mono; channel node shows binding); edges label conditions. *Right inspector*: fixed panel (or `Sheet` on narrow) with `Tabs` (Config · Mapping · Overrides), `Form` fields, the **CodeMirror DSL editor** with autocomplete, and inline `Alert` s from the live validator.
3. **Run timeline + live monitor** (the demo surface) — custom component: one row per agent, duration-scaled blocks (CSS width from ms), tool-call/inter-agent annotations as inline markers; `Tooltip` on hover (tokens/cost/latency in mono); click block → `Sheet` / `Dialog` with the full message thread (+ image thumbnails for multimodal); status `Badge` ; run header card with total cost/duration. Live block animation via the WS subscriber; `Skeleton` rows while connecting; failed/cost-breaker blocks in `destructive` .
4. **Eval pages** — `/evals` : `Table` of datasets + last-run pass-rate (inline sparkline) + `DropdownMenu` actions. `/evals/runs/{id}` : results `Table` (input | expected | actual | scores | pass `Badge` | rationale) with row → link to that example's **actual run timeline**. `Tabs` to compare last N runs.

Components to install (Phase 0)

`button` `card` `dialog` `sheet` `alert-dialog` `input` `textarea` `select` `switch` `slider` `label` `form` `tabs` `table` `badge` `dropdown-menu` `command` `popover` `tooltip` `scroll-area` `separator` `skeleton` `avatar` `sonner` (toasts).

Quality

Run `vercel:react-best-practices` over `.tsx` after each component batch (structure, hooks, a11y, perf, TS). Keep one accent color; no nested-cards-in-cards; `AlertDialog` (not `Dialog`) for destructive actions; designed empty/loading/error states everywhere.

Subsystem E — Structured handoffs & continuous learning

The differentiator that makes agents **stay coherent over days, not just minutes** (per the gstack "Structured handoffs" model). Two coupled mechanisms:

Structured handoffs (not freeform text)

When an agent finishes a step — and especially when it hands off to the next agent (workflow edge or `send_message_to_agent`) — it emits a `HandoffReport`, not just a blob of text:

```
HandoffReport = {
  output: Any,           # the actual artifact (lands at the node's output_key)
  implemented: str,      # what was accomplished
  outstanding: str,      # what was left undone / open questions
  actions: [{tool, input, result, status}], # commands/tools run + outcomes (from to
  issues: [str],         # problems discovered
  followed_procedures: bool, # did it stay within guardrails/instructions
}
```

- Persisted on the step (a `messages` row of role `agent` with the report as `tool_calls` /JSON) and **passed downstream** via `input_mapping`, so the next agent receives a structured briefing instead of "whatever the last one said."
- Rendered in the run timeline (the handoff is the click-through detail on a step) — this is what makes the multi-agent collaboration legible.
- Produced by a final "handoff" turn in the inner loop (a JSON-mode response validated by `JSONSchemaValidator`); falls back to `{output: <content>}` if the agent doesn't emit one.
- **Lands in Phase 5** as the structured form of inter-agent messaging.

Continuous-learning loop (New Task → Execute → Observe → Learn → Encode Skill)

Implemented through the extremis `ExternalMemoryStrategy`, closing the loop the brief's "openclaw SOUL.md/MEMORY" hints at:

- **New Task / prepare:** `memory_recall` — pull relevant episodic + procedural memories for this agent (its soul is the identity key) into the prompt.
- **Execute:** the run.
- **Observe:** capture the outcome (success/failure, cost, the `HandoffReport`, any guardrail trips).

- **Learn:** `memory_report_outcome` — record whether the approach worked, keyed to the agent's identity.
- **Encode Skill:** periodically `memory_consolidate` distills repeated tool-sequences into **procedural memory** ("how I did X") so future runs recall a learned skill, not just facts.
- **Lands in Phase 5** (memory subsystem); degrades gracefully when extremis is offline (skips recall/learn, runs stateless).

This elevates the platform from "multi-agent workflows" to "agents with a persistent soul that hand off coherently and get better with use" — the strongest interview narrative and directly on the extremis story.

Visibility surface (decision)

The **web run timeline is the primary monitoring UI** and must make collaboration legible: per-agent rows, **inter-agent edges** ("→ handoff to Analyst"), tool-call annotations, and click-through to the structured `HandoffReport` — this is the "what did it do / who did it talk to" surface. **Slack is documented future work** (not built now): it's purely additive on the existing `Channel` protocol + Redis event stream — (a) a real `SlackChannel` adapter and (b) a "run-mirror" that posts each run as a thread with agents as bot identities and handoffs as Block Kit blocks. Telegram remains the live human↔agent channel for the demo.

Data model

Base tables per `docs/architecture.md §4`: `agents`, `workflows`, `workflow_versions`, `channels`, `channel_bindings`, `runs`, `steps`, `messages`, `tool_invocations`, `schedules`, `outbound_messages` (costs denormalized message→step→run). Agents also carry `soul_md` + `persona` (identity layer, composed into the effective system prompt).

Schema reconciliation vs `architecture.md §4` (must land in the Phase 1 migration — code reads these):

- `channel_bindings.workflow_id UUID NULL` — webhook routing ("binding → workflow") needs it; arch DDL only had `agent_id`.
- `agents.default_workflow_id UUID NULL` — the binding fallback chain reads it.
- `agents.harness JSONB NOT NULL DEFAULT '{}'` — holds `{max_attempts, retry_on, validators[], interceptors[]}` for the config resolver (validators live here, **not** in `guardrails`).
- `workflows.current_version INT` pointer; **drop `workflows.graph`** — the graph lives only in `workflow_versions.graph` (no dual write). The `variables/nodes/edges` schema lives in `workflow_versions.graph`.

Harness/eval/media additions:

- `llm_attempts` (one row per retry: provider, request, raw_response, error, validation_failures, latency_ms — UI shows "succeeded on attempt 2/3").

- `llm_recordings` , `llm_recorded_calls` (sequence-ordered; image calls store a `media_assets` ref, not inline base64).
- `eval_datasets`, `eval_examples`, `eval_runs`, `eval_results` (deferred phase, but migration can land early).
- `media_assets` (image attachments) + `messages.attachments JSONB` referencing asset ids.

Alembic migrations for all.

Build phases (dependency order)

Reordered per the review so the frontend (60% of the grade lives there: demo 40% + UI/UX 20%) is **not** all back-loaded — a thin vertical UI slice lands at Phase 4 and gets iterated, not rushed.

- **Phase 0 — Scaffold/infra.** docker-compose (postgres, redis, backend, worker, frontend, code-runner, **extremis**). `backend/pyproject.toml` (fastapi, uvicorn, sqlalchemy[asyncio], asyncpg, alembic, pydantic-settings, langgraph, anthropic, openai, redis, apscheduler, lark, structlog, opentelemetry, python-telegram-bot, tavily-python, httpx, pyyaml, pytest). Frontend: Next.js + Tailwind + **shadcn init + tokens + app shell** + React Flow + CodeMirror. `.env.example` . Health endpoints (`/health/{db,redis,channels}`). **Milestone:** `docker-compose up` boots; health green; app shell renders.
- **Phase 1 — Data model & repositories.** All models + **the reconciled columns** (`channel_bindings.workflow_id` , `agents.default_workflow_id` , `agents.harness` , `workflows.current_version` , no `workflows.graph`) + Alembic initial migration + `repositories/` per aggregate. **Milestone:** migrations apply; CRUD repo tests pass.
- **Phase 2 — Harness thin core.** `HarnessExecutor` , `LLMProvider` protocol, `AnthropicProvider` + `StubProvider` + `ReplayProvider`, CostCap (conservative estimate) + Trace interceptors, JSONSchema + MaxLength validators, cost models (Decimal). **Milestone:** `test_executor` , `test_stub_provider` , `test_replay_provider` , stub-backed integration test pass.
- **Phase 3 — Two-layer runtime + minimal events.** `state.py` ; `inner_graph.py` (prepare→llm→router→{tool|end}, guardrail chokepoint, every transition persists steps/messages, **all LLM calls via harness**); `outer_graph.py` (dynamic compile from `workflow_versions.graph`); `dsl/` (Lark, no `eval`); `validation.py` ; `executor.py` (worker consumes from the **at-least-once** queue). Worker **emits run events to Redis post-commit** (minimal, so the UI can subscribe). **Milestone:** `test_workflow_execution` , `test_graph_validation` , `test_conditional_routing` .
- **Phase 4 — Thin UI vertical slice (pulled forward).** `lib/api.ts` (OpenAPI types) + `lib/ws.ts` . One **agent CRUD form** (schema-driven) and the **run timeline reading real WS events** from a stub-provider run. Demo surface is alive and iterable from here on. **Milestone:** create an agent in the UI → trigger a stub run → watch the timeline animate live.
- **Phase 5 — Tools, memory, guardrails.** Tools: `web_search` (Tavily), `http_request` (allowlist), `send_to_agent` (**run-per-message + inbox**), `send_to_channel`, `python_exec` (code-runner

over unix socket). Memory: buffer, summary, channel_scoped, **external→extremis (graceful degrade)**. Guardrails enforcer (CONTINUE/TERMINATE; **PAUSE / require_approval_for** is **designed-not-built** — resume needs the stubbed **human** node). **Milestone:** tool/memory/guardrail units + extremis round-trip + an inter-agent handoff producing a second run.

- **Phase 6 — Telegram (polling) + outbox + image ingestion.** **Channel** protocol; real **telegram.py** via **getUpdates** **long-polling** (no tunnel) — webhook handler also implemented + documented as production path; slack/whatsapp stubs; transactional outbox + dispatcher (backoff/retry); prompt-injection prefix. OpenAI provider + harness typed content blocks + image encoding; **media_assets** persistence; graceful-reply for unsupported media. **Milestone:** **test_channel_roundtrip** ; live bot reply; photo → agent describes it; voice note → graceful reply.
- **Phase 7 — Scheduler + full real-time.** APScheduler; WS gateway forwarding + **backpressure + replay from messages** ; structlog context binding; OTel→optional Jaeger. **Milestone:** run streams live; reconnect replays cleanly; cron-triggered run fires.
- **Phase 8 — Full frontend.** React Flow builder (palette, canvas, inspector, DSL editor, live validation, dry-run highlight); versioned workflow saves; channels UI; dashboard polish + the one timeline reveal animation. **vercel:react-best-practices** over each **.tsx** batch. **Milestone:** create agent → build 2-agent workflow → trigger via Telegram → watch timeline → see cost.
- **Phase 9 — Deepen harness + eval (deferred / first cut).** Bedrock provider; CitationValidator; CompositeJudge; script Jinja2/ **include:** ; recording CLI; eval data model + LLMJudge + runner + minimal eval UI; 10-example datasets per template. **Milestone:** record→replay roundtrip byte-identical; **harness eval run** produces results. *Skip if Phase 8 demo isn't solid.*
- **Phase 10 — Templates, seed, docs, demo.** Two templates + **scripts/seed.py** ; README per **docs/architecture.md §16** (Harness chapter, workflow chapter, **scope-vs-cut table**, failure modes); **docs/{adding-a-channel,adding-a-tool,adding-a-template,workflow-dsl}.md** . Playwright golden-path E2E. **Record demo in replay mode by ~day 10, two takes.**

Files to create (representative)

- Runtime: **backend/app/runtime/{state,inner_graph,outer_graph,executor,validation}.py** , **runtime/dsl/{grammar.lark,parser.py,evaluator.py}**
- Harness: **backend/app/harness/{executor,call,config,cli}.py** , **harness/providers/{base,anthropic,openai,bedrock,stub,replay}.py** , **harness/validators/*** , **harness/interceptors/*** , **harness/cost/{models,pricing}.py** , **harness/recording/{recorder,replayer,script_generator}.py** , **harness/eval/{runner,reports}.py** + **harness/eval/judges/{base,exact_match,llm_judge,composite}.py**

- Persistence: `backend/app/db/models.py` , `db/repositories/{agents,workflows,runs,channels,evals}.py` , `alembic/versions/*`
- Seams: `channels/{base,telegram,slack,whatsapp,registry}.py` , `tools/*` , `memory/{base,buffer,summary,channel_scoped,external}.py` , `guardrails/{enforcer,policies}.py`
- Frontend: `components/{workflow-builder,timeline,agent-config,evals,ui}/` , `lib/{api,ws}.ts`
- Infra/docs: `docker-compose.yml` , `code-runner/{Dockerfile,runner.py}` , `scripts/{seed,reset_db}.py` , `tests/scripts/*.yaml` , `README.md` , `docs/*.md` , `.env.example`

Reuse from existing code

`~/agentwork` is a different stack — **patterns only**: Anthropic SDK usage and the `extremis` client `init ( Extremis(Config(namespace=...)) )` for `memory/external.py`. Treat as greenfield.

Scope & sequencing note

Large build (~14 days). The harness is built **thin-first** (Phase 2 core) and **deepened only in Phase 9**, so it can't starve the frontend the way "harness = ~30% of budget up front" would. **Cut order** if time runs short (preserving demo 40% + architecture 30%): whole **eval framework** → `parallel` node → script Jinja2/ `include:` → Bedrock provider → CompositeJudge → CitationValidator → recording CLI (keep record/replay itself). **Never cut**: the two-layer runtime, harness thin core (executor + Anthropic/Stub/Replay + CostCap), at-least-once queue, Telegram round-trip (polling), the live timeline (lands Phase 4), cost circuit breaker, two templates, ChannelScoped memory. Keep the "**scope vs cut**" table in the **README** — it signals judgment and pre-answers half the live-session questions.

Open decision for the user: the reviewer recommended *cutting* harness depth outright; I instead **re-sequenced** it (thin-first + deferred deep parts) to honor the locked "everything built." If you'd rather hard-cut the eval framework / Bedrock / composite judge entirely (not just defer), say so and I'll delete those sections rather than schedule them.

Verification (end-to-end, in order)

1. `cp .env.example .env` (fill keys) → `docker-compose up` → health endpoints green.
2. `python scripts/seed.py` → two templates + sample agents in UI.
3. `pytest backend/tests` — integration (`agent_lifecycle` , `workflow_execution` , `channel_roundtrip` , `conditional_routing` , `workflow_versioning`) + harness

- (`executor` , `stub/replay providers` , `validators` , `cost_models` Decimal, `recording` roundtrip, `eval_runner`) + DSL + guardrails + extremis round-trip.
4. Playwright E2E: create agent → run workflow → timeline renders expected steps.
 5. **Live Telegram (polling, no tunnel):** start the bot (`getUpdates`), message the bound agent, watch the run on the timeline, get a reply. (The 40%.) Multi-turn continuity verified via `ChannelScopedMemory` across two separate messages/runs.
 6. **Guardrail demo:** set `max_cost_per_run_usd=$0.10` ; confirm the conservative pre-call estimate trips the breaker, graceful termination, red timeline block, `runs.error` set. 6c. **Crash-safety:** kill a worker mid-run; confirm the at-least-once queue re-delivers and the run completes (or is marked failed via heartbeat), not lost. 6b. **Image round-trip:** send a photo via Telegram → agent (vision model) describes it; thumbnail renders in the timeline. Send a voice note → graceful "can't process that yet" reply.
 7. **Deterministic demo:** `LLM_MODE=record` once, then `LLM_MODE=replay` reproduces the run offline at zero API cost.
 8. **Eval:** `harness eval run market_intel_briefing` → results table populated, click a failed example → lands on its actual run timeline.
 9. **Extensibility proof:** diff to add the Slack stub (only `channels/slack.py` + registry line + UI form) — no orchestrator/agent/workflow changes; diff to add a provider (only `providers/<x>.py` + registry).

Primary risk

Not technical — the **demo recording**. Two structural de-risks now baked in: **Telegram long-polling** (no public tunnel to fail mid-take) and **replay mode** (capture once, replay deterministically, offline, zero API cost). extremis ships in-compose and degrades gracefully, so neither the "single command" nor the offline demo depends on a reachable external. Script the narrative before building, record by ~day 10, two takes. If a feature can't earn screen time, question whether it earns build time.